

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

**ОТЧЁТ
ПО УЧЕБНОЙ ПРАКТИКЕ**

Тема: ГЕНЕТИЧЕСКИЕ АЛГОРИТМЫ

Студент гр. 4344

Студент гр. 4341

Студент гр. 4341

Руководитель

А.И. Бегеза

В.А. Будкин

Д.А. Мигрин

Т.Р. Жангиров

Санкт-Петербург

2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студенты: А.И. Бегеза, В.А. Будкин, Д.А. Мигрин

Группа: 4344, 4341, 4341

Тема практики: Генетические алгоритмы

Задание на практику:

Задача о назначениях

Пусть имеется N работ и N кандидатов на выполнение этих работ, причем назначение j -й работы i -му кандидату требует затрат $c_{ij} > 0$. Необходимо назначить каждому кандидату по работе, чтобы минимизировать суммарные затраты. Причем каждый кандидат может быть назначен на одну работу, а каждая работа может выполняться только одним кандидатом.

Сроки прохождения практики: 06.07.2026 – 17.07.2026

Дата сдачи отчета: 17.07.2026

Дата защиты отчета: 17.07.2026

Студент	_____	А.И. Бегеза
Студент	_____	В.А. Будкин
Студент	_____	Д.А. Мигрин
Руководитель	_____	Т.Р. Жангиров

АННОТАЦИЯ

Работа посвящена решению задачи о назначениях методом генетических алгоритмов, визуализации решения и промежуточных шагов, а также созданию графического интерфейса для удобного взаимодействия с программой.

SUMMARY

The work is about solving assignment problem by genetic algorithms method: solution and intermediate steps visualisation, and also about developing graphical user interface.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	5
1 ПОСТАНОВКА ЗАДАЧИ.....	6
1.1 Предметная область.....	6
1.2 Задачи практики.....	6
1.3 Члены бригады.....	6
2 РЕЗУЛЬТАТЫ РАБОТЫ.....	8
2.1. Распределение зон ответственности.....	8
2.2. Изучение генетических алгоритмов как подхода к решению задач.....	8
2.3. Создание модели решения задачи на языке генетических алгоритмов.....	8
2.4. Создание шаблона графического интерфейса.....	10
2.5. Реализация графического интерфейса.....	12
2.6. Создание программы на языке программирования C++.....	14
2.7. Визуализация решения задачи.....	17
2.8. Связь программы и графического интерфейса.....	24
2.9. Демонстрация финальной версии программы и отчета.....	25
3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ.....	26
4 ОТЗЫВ РУКОВОДИТЕЛЯ.....	27
ЗАКЛЮЧЕНИЕ.....	28
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	29

ВВЕДЕНИЕ

Решение поставленного задания применимо в различных предметных областях: логистике, планировании, производстве, кадровом менеджменте.

Целью учебной практики является решение задания методом генетических алгоритмов и создание графического интерфейса для удобного пользования результатами проделанной работы.

В задачи практики входят изучение метода генетических алгоритмов, создание программы, графического интерфейса, их объединение и тестирование; демонстрация работы.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

Решенная в рамках практики задача позволяет решать проблемы в следующих областях:

- Планирование: распределение потоков в многопроцессорных системах;
- Кадровый менеджмент: распределение сотрудников по должностям с учетом их навыков и зарплатных ожиданий;
- Логистика: назначение транспортных средств на различные маршруты или заказы;
- Производство: привязка производственных задач к конкретному оборудованию.

1.2 Задачи практики

Напишите список задач, поставленных перед вами в рамках практики. Не забудьте указать пункты по изучению новых технологий, которые вам понадобились для работы в рамках практики.

В рамках практики были поставлены следующие задачи:

- Изучение генетических алгоритмов как подхода к решению задач;
- Создание модели решения задачи на языке генетических алгоритмов;
- Создание шаблона графического интерфейса;
- Реализация графического интерфейса;
- Создание программы на языке программирования C++;
- Визуализация решения задачи;
- Связь программы и графического интерфейса;
- Тестирование финальной версии программы;
- Демонстрация финальной версии программы и отчета.

1.3 Члены бригады

В.А. Будкин выполнял задачи: создание модели решения задачи, разработка программы алгоритма на языке программирования C++.

Д.А. Мигрин выполнял задачи: создание шаблона графического интерфейса, создание графического интерфейса.

А.И. Бегеза выполнял задачи: визуализация решения задачи, связь программы и графического интерфейса.

2 РЕЗУЛЬТАТЫ РАБОТЫ

В ходе работы были распределены роли и решены все поставленные задачи.

2.1. Распределение зон ответственности

В.А. Будкин: Создание модели решения задачи, разработка программы алгоритма на языке программирования C++.

Д.А. Мигрин: Создание шаблона графического интерфейса, создание графического интерфейса.

А.И. Бегеза: Визуализация решения задачи, связь программы и графического интерфейса.

2.2. Изучение генетических алгоритмов как подхода к решению задач.

Для достижения цели был изучен метод решения задач генетическими алгоритмами с помощью методических материалов и лекции преподавателя. Благодаря подходу этого метода была составлена модель решения, описанная ниже.

2.3. Создание модели решения задачи на языке генетических алгоритмов.

Для решения задачи о назначениях была разработана модель на основе генетического алгоритма. В задаче требовалось назначить каждому работнику ровно одну работу таким образом, чтобы каждая работа была назначена только одному работнику, а суммарная стоимость назначений была минимальной.

Матрица затрат имеет размер $N \times N$. Строки матрицы соответствуют работникам, а столбцы соответствуют работам. Элемент $c[i][j]$ показывает стоимость выполнения j -й работы i -м работником..

Хромосома, представляется перестановкой чисел от 0 до $N - 1$, где N - количество работников и работ. Индекс элемента хромосомы соответствует номеру рабочего, а значение элемента соответствует номеру назначенной ему работы.

Стоимость одного решения вычисляется как сумма затрат по всем назначениям работ:

$$\text{cost} = c[0][\text{chromosome}[0]] + c[1][\text{chromosome}[1]] + \dots + c[N-1][\text{chromosome}[N-1]]$$

Так как в задаче нужно минимизировать затраты, оптимальным считается решение с меньшим значением cost.

Для работы методов отбора дополнительно используется значение приспособленности fitness. Оно считается по формуле: $\text{fitness} = 1 / (1 + \text{cost})$.

Начальная популяция создаётся случайным образом. Для каждой особи сначала создаётся массив работ от 0 до N - 1, после чего этот массив случайно перемешивается. Поэтому каждая особь сразу является корректным решением задачи, т.е. не могут встретиться работы с одинаковыми номерами.

В программе реализованы два метода отбора родителей: турнирный отбор и стохастическая универсальная рулетка.

При турнирном отборе случайно выбирается несколько особей из популяции. Количество таких особей задаётся параметром tournamentSize. Из выбранных особей родителем становится та, у которой стоимость решения меньше.

Стохастическая рулетка работает иначе. Сначала считается сумма приспособленностей всех особей(S). Далее задаётся количество указателей(стрелок на колесе) r. Расстояние между соседними указателями определяется как: $\text{step} = S / r$. Первая позиция указателя выбирается случайным образом на отрезке от 0 до step. Остальные указатели располагаются через step. Каждый указатель выбирает ту особь, в чей участок он попал. Такой метод уменьшает случайные перекосы обычной рулетки, потому что за один запуск выбирается сразу несколько родителей из относительно разных областей колеса.

Для скрещивания реализованы два метода: упорядоченное скрещивание и позиционное скрещивание.

При упорядоченном скрещивании случайно выбирается участок хромосомы первого родителя. Этот участок копируется в ребенка. Остальные позиции заполняются генами второго родителя в том порядке, в котором они у него идут. Уже использованные работы пропускаются.

При позиционном скрещивании часть позиций случайно берется от первого родителя. После этого оставшиеся пустые места заполняются работами второго родителя, которые еще не были использованы.

Для изменения отдельных особей реализованы два метода мутации: мутация обменом и мутация инверсией. При мутации обменом выбираются два случайных гена хромосомы, после чего они меняются местами.

При мутации инверсией выбирается случайный участок хромосомы, после чего порядок элементов внутри этого участка разворачивается.

Также в алгоритме используется элитизм, несколько особей с наибольшим числом приспособленности из текущего поколения напрямую переходят в следующее поколение без изменений. Это нужно для того, чтобы уже найденные оптимальные решения не терялись после скрещивания или мутации.

Алгоритм завершает работу в двух случаях: если достигнуто заданное количество поколений или если в течение заданного числа поколений не было улучшения оптимального найденного решения.

2.4. Создание шаблона графического интерфейса.

С помощью библиотеки `gaulib` был создан шаблон графического интерфейса. Все элементы управления построены на основе двух классов: `Button` (кнопка) и `NumberBox` (текстовое поле для ввода чисел). Класс `Button` отвечает за отрисовку прямоугольной области с текстом, изменяет цвет при наведении курсора и обрабатывает нажатие кнопки мыши. Класс `NumberBox` обеспечивает ввод целочисленных значений с ограничением по минимальному и максимальному диапазону, автоматически корректирует вводимые данные, поддерживает клавиши `Backspace`, `Enter`, а также переключение активного поля по щелчку мыши с изменением курсора.

Для переключением между различными окнами интерфейса создана структура `Screen`, определяющая четыре состояния (`EXIT`, `MENU`, `MTX_CREATION`, `VISUALIZATION`), а функции `menu_screen`, `mtx_creation_screen` и `visualization_screen` отвечают за отрисовку и обработку событий на каждом из экранов. Все элементы интерфейса обновляются и отрисовываются в каждом кадре основного цикла приложения. При ошибках (например, при попытке загрузить некорректный файл) текст ошибки сохраняется в глобальной переменной `errorMessage` и выводится внизу экрана.

В главном меню расположены кнопки для выбора способа считывания/создания матрицы, а также место для полей настроек (рис. 1).

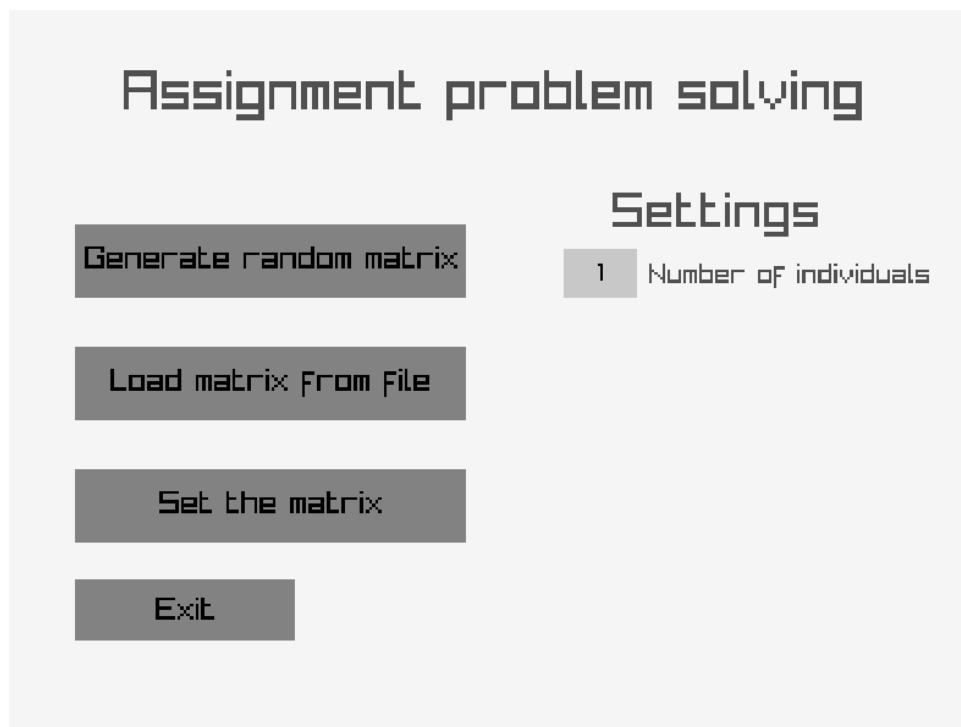


Рисунок 1 — шаблон графического интерфейса.

2.5. Реализация графического интерфейса.

Настройка алгоритма с помощью графического интерфейса проходит в главном меню (рис. 2).

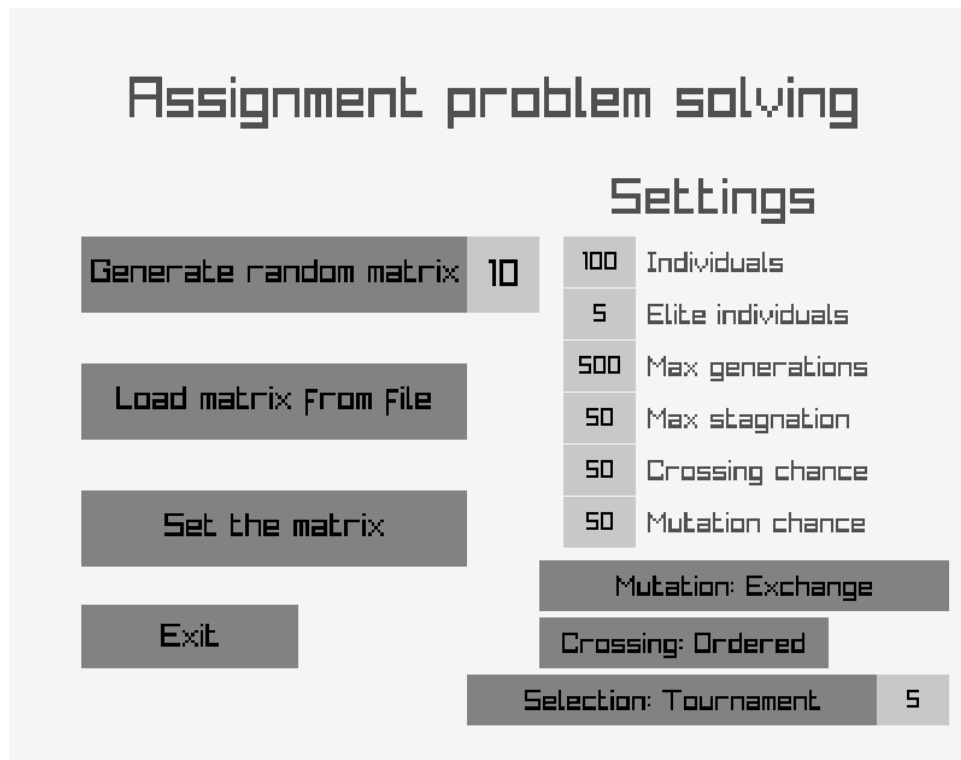


Рисунок 2 — главное меню.

В качестве параметров алгоритма можно настроить количество особей, количество элитных особей, максимальное число поколений и максимальное число стагнирующих поколений для остановки, шанс скрещивания и шанс мутации. Также предусмотрены кнопки с циклическим переключением для выбора метода мутации, метода скрещивания и метода отбора с настраиваемым значением.

Все текстовые поля автоматически синхронизируют свои значения: максимальное количество элитных особей ограничено размером популяции, а число стагнирующих поколений — максимальным числом поколений. Ввод чисел защищён от выхода за допустимые пределы: при вводе значения выше максимума поле автоматически устанавливает максимальное, при выходе из поля значения ниже минимума заменяются на минимальное.

В качестве способов задания матрицы были реализованы следующие способы: создание случайной матрицы, загрузка матрицы из файла, задание матрицы вручную.

При выборе способа чтения матрицы из файла вызывается диалог выбора файла с помощью библиотеки `tinyfiledialogs`. Реализован фильтр расширений — разрешены только файлы формата `*.csv`. При выборе корректного файла матрица считывается, проверяется на наличие ошибок и, если всё верно, выполняется переход к экрану визуализации. В случае ошибки в нижней части главного меню отображается сообщение «Invalid file format» красным цветом (рис. 3).

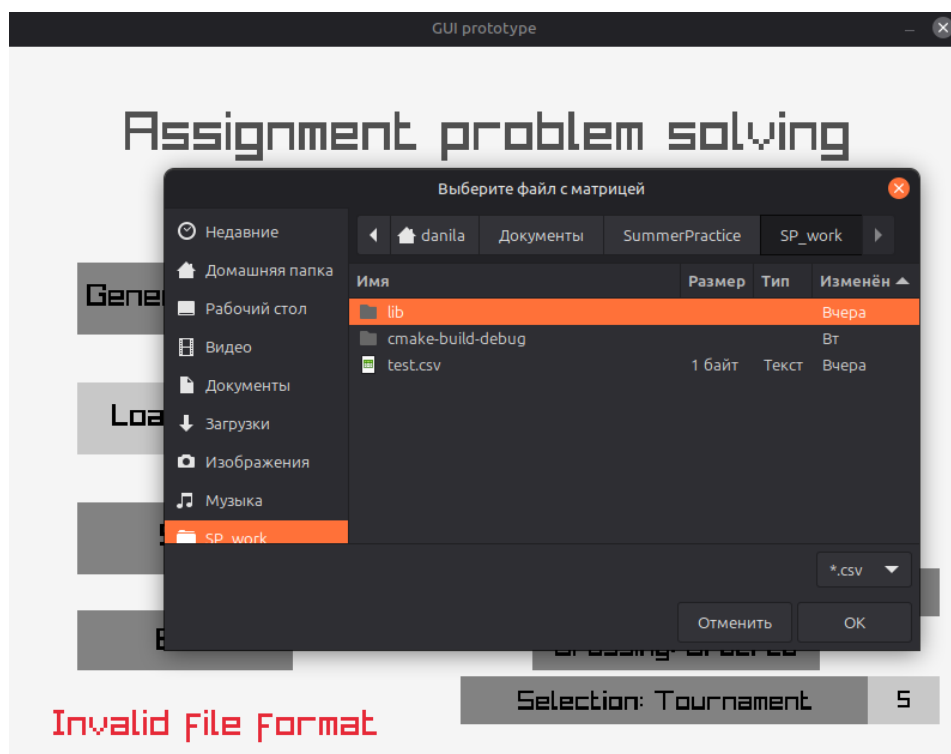


Рисунок 3 — окно выбора файла и отображение ошибки.

При выборе способа задания матрицы вручную предлагается выбрать размер матрицы, которая по умолчанию заполнена единицами, а значение каждой ячейки можно изменять (рис. 4).

GUI prototype

Back

Cost matrix creation

10 Matrix size

Works

		0	1	2	3	4	5	6	7	8	9
0		43	7	2	7	8	1	8	6	3	1
1		235	3	1	46	5	7	8	2	843	59
2		23	46	57	42	5	7	5	4	335	454
3	Ш	6	6	6	6	6	6	6	6	6	6
4	о	324	92	120	20	350	44	3	53	82	44
5	г	23	5	6	235	36	864	4	345	46	34
6	к	2	11	1	23	5	64	46	82	347	5
7	е	32	46	74	74	23	644	73	24	64	234
8	с	45	64	48	2	6	88	6	6	55	55
9		7	7	7	7	7	7	7	7	7	7

Continue

Рисунок 4 — окно задания матрицы.

При изменении размера динамически перестраивается сетка из ячеек, каждая из которых является текстовым полем для ввода числа. Для удобства ввода реализована навигация: при нажатии клавиши Enter фокус автоматически переходит на следующую ячейку. Строки и столбцы матрицы подписаны номерами для удобства чтения. После заполнения всех значений пользователь нажимает кнопку «Continue», и матрица передаётся в алгоритм, после чего открывается экран визуализации.

2.6. Создание программы на языке программирования C++.

Генетический алгоритм был реализован на языке C++. Основная логика алгоритма вынесена в класс GeneticAlgorithm. Данный класс отвечает за создание начальной популяции, оценку особей, выполнение отбора, скрещивания, мутации, формирование новых поколений и возврат итогового результата.

Для хранения одной особи используется структура Individual. В ней хранится хромосома, стоимость решения (cost) и значение приспособленности (fitness). Хромосома является вектором целых

чисел(vector). Индекс элемента соответствует номеру работника, а значение элемента соответствует номеру назначенной работы.

Настройки алгоритма хранятся в структуре `GASettings`. Через неё можно задавать размер популяции `populationSize`, количество поколений `generations`, вероятность скрещивания `crossoverProbability`, вероятность мутации `mutationProbability`, размер турнира `tournamentSize`, количество элитных особей `eliteCount`, условие остановки без улучшения `maxGenerationsWithoutImprovement`, количество указателей для стохастической рулетки `roulettePointerCount`, тип отбора `selectionType`, тип мутации `mutationType` и тип скрещивания `crossoverType`.

Тип отбора задаётся перечислением `SelectionType`. На данном этапе доступны турнирный отбор `TournamentSelection` и стохастическая универсальная рулетка `RouletteSelection`. Тип мутации задаётся перечислением `MutationType`, где доступны мутация обменом `SwapMutation` и мутация инверсией `InversionMutation`. Тип скрещивания задаётся перечислением `CrossoverType`, где доступны упорядоченное скрещивание `OrderedCrossover` и позиционное скрещивание `PositionalCrossing`.

Перед запуском алгоритма настройки проверяются методом `validateSettings`. Проверяется размер популяции, количество поколений, допустимость вероятностей скрещивания и мутации, количество элитных особей, размер турнира и параметры остановки по стагнации.

Результат работы алгоритма хранится в структуре `GAResult`. В ней находится оптимальное найденное решение `bestIndividual` и история поколений `history`.

Информация об одном поколении хранится в структуре `GenerationInfo`. Она содержит номер поколения `generationNumber`, полную популяцию `population`, количество особей `individualCount`, особь с наибольшим числом приспособленности `bestIndividual` и стоимость этой особи `bestCost`. Популяция сохраняется в отсортированном виде (от оптимального решения к неоптимальному).

Перед запуском алгоритма считывается матрица затрат `costMatrix`. Для этого используется класс `MatrixReader`. Он открывает CSV-файл, считывает строки матрицы, разбивает строки на отдельные значения, преобразует значения в целые числа и проверяет корректность данных. Проверяется, что значения не пустые, являются числами больше нуля, а сама матрица является квадратной.

После чтения матрицы создаются настройки алгоритма `settings`. Затем создаётся объект класса `GeneticAlgorithm`, и вызывается метод `run`. Метод `run` получает матрицу затрат `costMatrix` и настройки алгоритма `settings`, после чего запускает основной цикл ГА.

Внутри метода `run` сначала создаётся начальная популяция с помощью метода `createInitialPopulation`. Для создания отдельной случайной особи используется метод `createRandomIndividual`, а для создания случайной хромосомы - метод `createRandomChromosome`. После создания особи для неё вычисляются стоимость и приспособленность с помощью метода `evaluateIndividual`.

Стоимость решения вычисляется методом `calculateCost`. Значение приспособленности вычисляется методом `calculateFitness`. Поиск особи с наибольшим числом приспособленности в популяции выполняется методом `getBestIndividual`.

На каждом поколении алгоритм сохраняет в `history` номер поколения, полную отсортированную популяцию, количество особей, особь с наибольшим числом приспособленности и её стоимость. Также отслеживается оптимальное решение за всё время работы алгоритма `globalBest`. Если текущее оптимальное решение не улучшается в течение заданного количества поколений, работа алгоритма может быть завершена досрочно.

Формирование нового поколения выполняется методом `createNextGeneration`. В начале текущая популяция сортируется по стоимости решения. Затем несколько особей с наибольшим числом приспособленности

переносятся в новое поколение без изменений в соответствии с параметром `eliteCount`. После этого оставшаяся часть поколения заполняется потомками.

Родители для потомков выбираются в зависимости от выбранного типа отбора `selectionType`. При турнирном отборе используется метод `tournamentSelection`. При использовании стохастической универсальной рулетки применяется метод `stochasticRouletteSelection`, который выбирает набор родителей с помощью нескольких равномерно расположенных указателей.

Скрещивание выполняется с вероятностью `crossoverProbability`. В зависимости от выбранного типа скрещивания `crossoverType` вызывается метод `orderedCrossover` или `positionCrossover`. Если скрещивание не выполняется, потомком становится копия одного из родителей.

Мутация выполняется с вероятностью `mutationProbability`. В зависимости от выбранного типа мутации `mutationType` вызывается метод `mutateSwap` или `mutateInversion`. После скрещивания и мутации потомок заново оценивается методом `evaluateIndividual` и добавляется в новое поколение.

После завершения работы программа возвращает объект `GAResult`, содержащий оптимальное решение `bestIndividual` и историю поколений `history`. По итоговой хромосоме можно восстановить назначение работ работникам, а значение `cost` показывает суммарные затраты найденного решения.

2.7. Визуализация решения задачи.

Визуализация решения задачи выполнена в виде графика, показывающего зависимость оптимального решения задачи от номера популяции, в виде динамически изменяющейся таблицы соответствия работника, работы и стоимости этой работы у данного работника, а также в виде полей, показывающих текущий шаг алгоритма и текущее оптимальное решение (рис. 5).

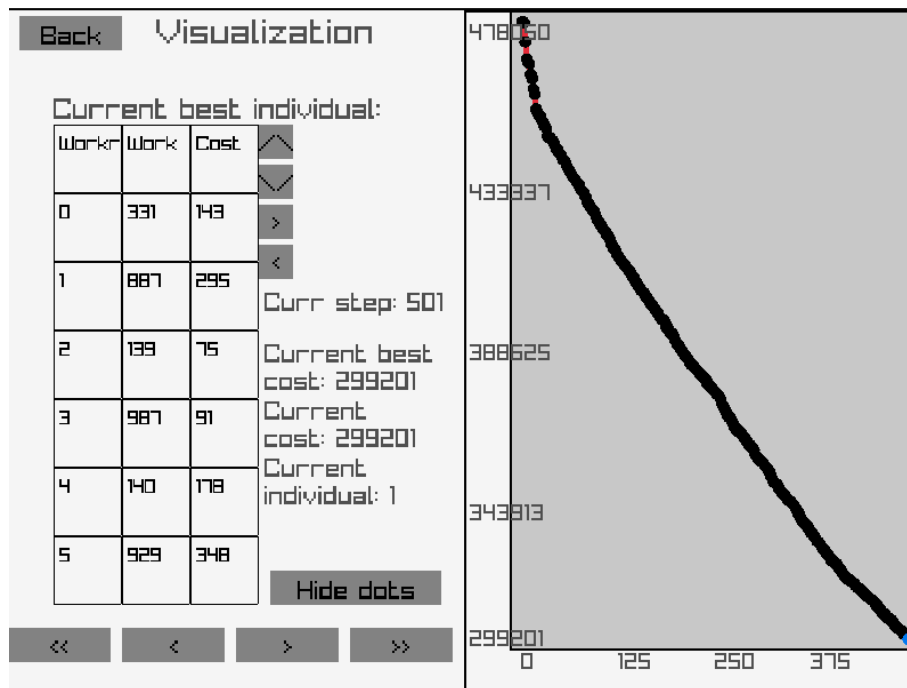


Рисунок 5 — визуализация решения задачи.

Для построения таблицы был создан класс Table, содержащий параметры таблицы (количество строк, количество столбцов, ширина и высота клетки, текущая страница для отображения) и данные каждой ячейки в виде `std::vector<std::vector<std::string>> data`. Так как число N может быть достаточно большим, были добавлены элементы управления таблицей — кнопки с изображением стрелок с левой стороны таблицы. Они позволяют просматривать следующие элементы особи (рис. 6).

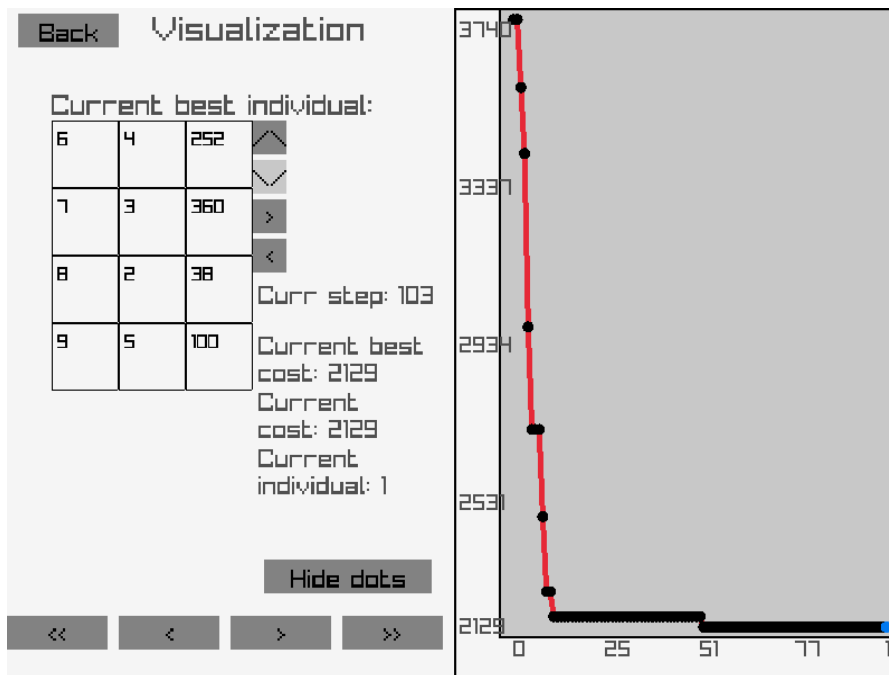


Рисунок 6 – Переключение ячеек таблицы

Данный механизм реализован с помощью параметра `currentPage`. Он позволяет отображать ограниченное число ячеек таблицы. Для этого в методе отображения таблицы `Table::draw(int x, int y)`, используются следующие расчеты: позиция левого верхнего угла клетки i, j таблицы рассчитывается как $\text{int } X_{\text{cell}} = x + i * \text{cellWidth}$, $\text{int } Y_{\text{cell}} = y + j * \text{cellHeight}$, где x, y — координаты левого верхнего угла таблицы, i — столбец клетки, j — строка клетки. Затем по найденным координатам строится рамка с помощью библиотечной функции `DrawRectangleLines`, а внутри этой рамки записывается число `data[currentPage*visiblePages + i][j]`.

Для переключениями между страницами таблицы используются кнопки. При нажатии на кнопку поле класса `Table` `currentPage` либо повышается на единицу (при нажатии на правую кнопку от таблицы), либо понижается на единицу (при нажатии на левую). Внутри методов, отвечающих за изменение параметра `currentPage`, `nextPage` и `prevPage` реализованы проверки на выход за пределы `data`.

Таблица формируется один раз при запуске окна `visualization`, когда пользователь делает первый шаг. Если программа отображает следующий

или предыдущий шаг, ячейки таблицы обновляются в соответствии с текущим шагом визуализации currStep.

Также была добавлена возможность просмотра всех особей популяции с помощью соседних кнопок со стрелками влево и вправо. Они позволяют просматривать результаты всех особей (рис. 7). Реализовано с помощью дополнительной переменной currIndividual, которая показывает номер текущей особи в популяции.

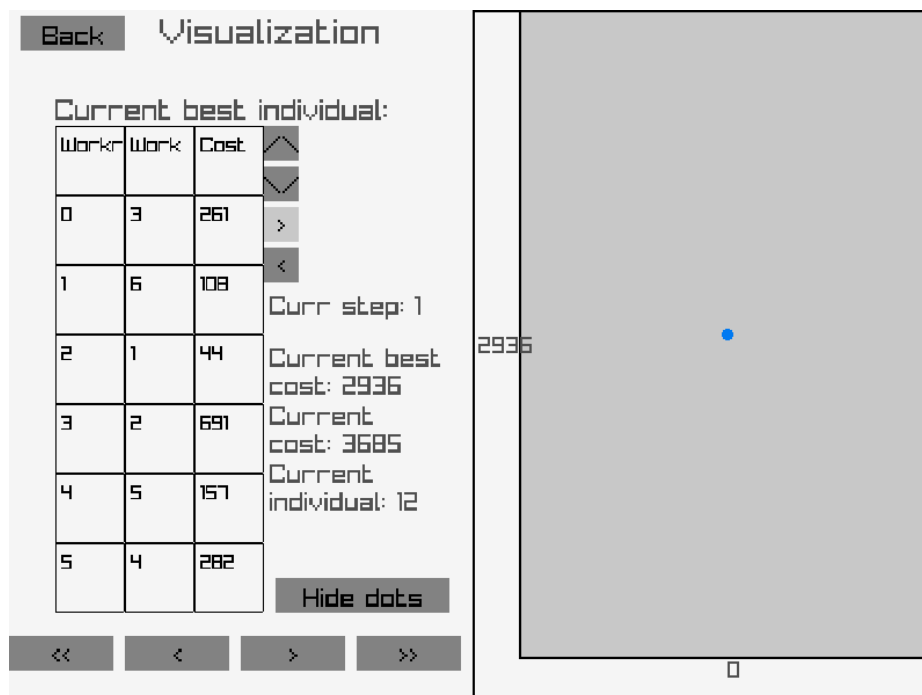


Рисунок 7 — переключение между особями в таблице.

Таблица изменяется по мере отображения следующих или предыдущих шагов. Для переключения отображения шагов алгоритма используются кнопки внизу экрана. Элементы «<» и «>» позволяют визуализировать один шаг алгоритма, элемент «>>» позволяет пропустить все шаги алгоритма и сразу показать результат.

При нажатии на кнопку «>» вызывается вспомогательная функция `void addPoint(std::vector<Vector2>& points, const GAResult& data, const Rectangle& graphArea, Scaling& scale, int& currStep)`, где `points` — вектор точек для построения графика, `data` — результаты работы алгоритма, `graphArea` — зона, в которой строится график, `scale` — текущие максимальные и минимальные

значения рассмотренных шагов алгоритма для масштабирования графика, currStep — текущий шаг алгоритма. В данной функции берется новая точка из результата работы алгоритма, соответствующая текущему шагу алгоритма currStep и добавляется в общий вектор точек. Затем обновляются максимальные и минимальные значения scale для будущего масштабирования. После чего пересчитываются координаты уже найденных точек с учетом новой добавленной точки с целью читаемости, а значение currStep поднимается на единицу. И так как алгоритм сделал шаг, нужно обновить значения в ячейках таблицы, и вызывается вспомогательный метод updateTable, обновляющий значения ячеек таблицы в соответствии с текущим шагом currStep.

Кнопка «>>» выполняет все то же, что и кнопка, описанная выше, но до тех пор, пока не будут использованы все значения из GAResult data.

Элемент «<» вызывает вспомогательную функцию removePoint, удаляющая последнюю точку из вектора points и пересчитывающая координаты оставшихся точек с учетом удаленных минимальных и максимальных значений последней точки. Значение currStep понижается на единицу. Затем обновляется таблица с отображением особей.

Построение графика начинается с выделения для него зоны. При вызове функции addPoint точка добавляется в вектор точек, а затем весь вектор масштабируется (рис. 8 и рис. 9). Для этого находятся коэффициенты, на которые домножаются текущие координаты точки

$$xScale = (graphArea.width - 20) / (float)(scale.maxX - scale.minX + 1);$$
$$yScale = (graphArea.height - 20) / (float)(scale.maxY - scale.minY + 1).$$

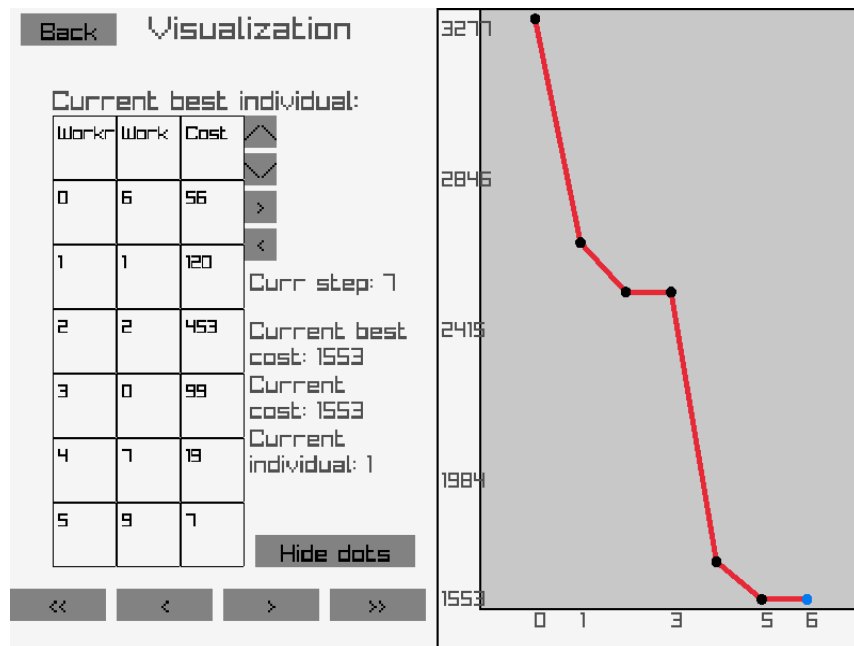


Рисунок 8 — график до пересчета точек.

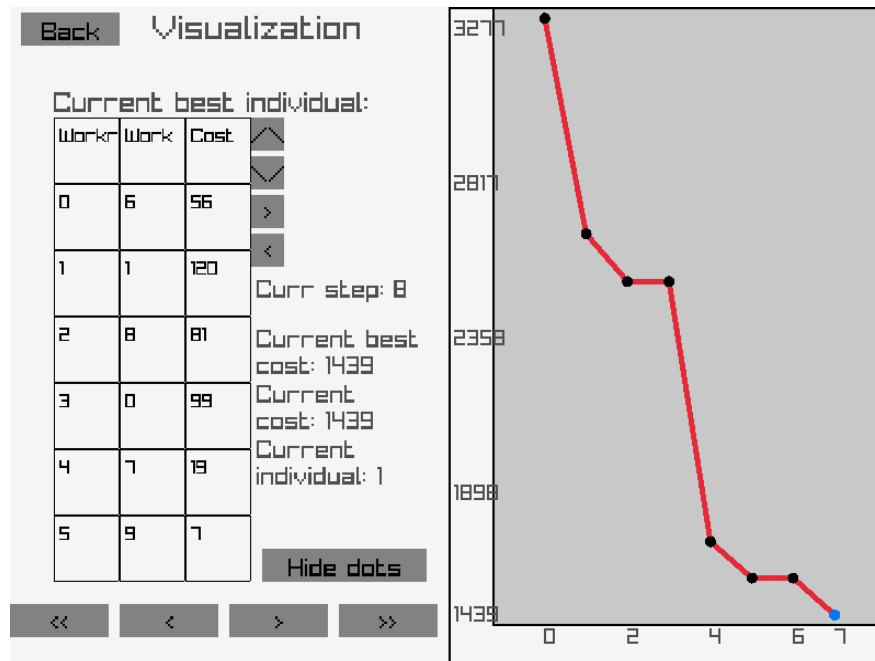


Рисунок 9 — график после пересчета точек.

Данная формула позволяет скорректировать положение точки на графике с учетом размера графика и разницы максимального и минимального значения точек вообще. Координаты точек пересчитываются следующим образом:

$$\text{points}[i].x = \text{graphArea}.x + 10 + (x - \text{scale}.minX) * xScale;$$

$$\text{points}[i].y = \text{graphArea}.y + \text{graphArea}.height - 10 - (y - \text{scale}.minY) * yScale,$$

где x, y — изначальные координаты точки, $graphArea.x$, $graphArea.y$ — координаты левого верхнего угла зоны графика. Вычитание из изначальных координат точек и позволяет находить новые координаты в зависимости от минимального и максимального значения. Если добавлена всего одна точка, то добавлен расчет координат для нее отдельно, так как в этом случае значение $minY$ и $maxY$ совпадает.

Таким образом формируется вектор точек, по которым строится график с помощью библиотечной функции `DrawLineEx`.

Построение делений на оси графика реализовано с помощью вычисления равных интервалов и шагов для соответствующих осей. Для оси абсцисс вычисляются деления $division = std::floor(scale.minX + i * xStepValue)$, где i — шаг из числа делений, а $xStepValue = (scale.maxX - scale.minX) / (float)(divisionsNumber - 1)$ определяет единичный отрезок для графика. Далее под графиком в месте, соответствующем координатам $(points[division].x, graphArea.y + graphArea.height)$, где $points[division].x$ координата x ближайшей снизу точки к найденной точке деления.

Иным образом высчитываются координаты для делений на оси ординат, а именно: $(x = WholeGraphArea.x, y = graphArea.y + i * yInterval + 10)$, а $division = scale.maxY - i * yStepValue$, где $yStepValue$ считается аналогично $xStepValue$, $yInterval = graphArea.height / (divisionsNumber - 1)$, $WholeGraphArea.x$ — координата x области графика с учетом места для делений.

Также была добавлена кнопка для отображения точек, так как при большом количестве поколений точек становится слишком много (рис. 10).

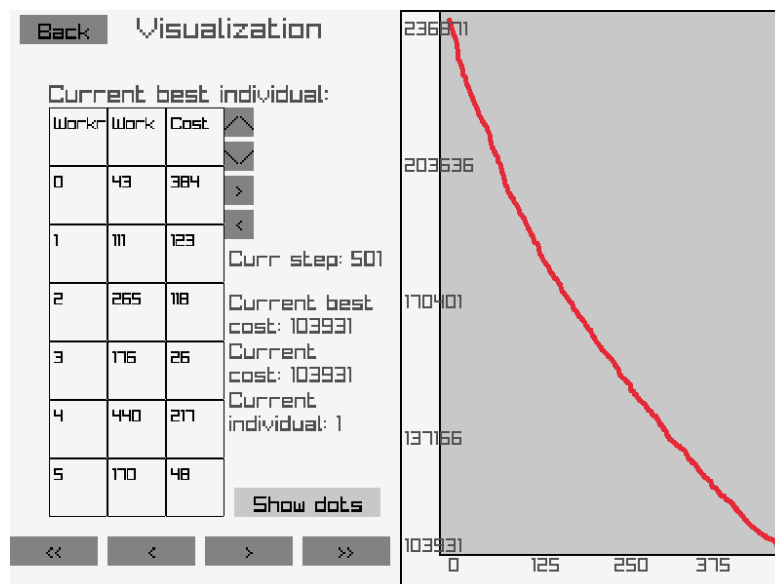


Рисунок 10 — график после пересчета точек.

2.8. Связь программы и графического интерфейса.

Для связи программы и графического интерфейса был реализован класс GUI, методы и часть полей которого отвечают за визуализацию решения и графический интерфейс, а другая часть полей (`std::vector<std::vector<int>>` `costMatrix`, `GASettings settings`, `GAResult result`) отвечает за хранение данных и настроек для работы алгоритма.

Возможность настройки алгоритма реализована в главном меню (рис. 2). Параметры, выбранные пользователем передаются в уже описанной функции `menu_screen`. Поля `number` ячеек класса `NumberBox` записываются в поле класса `settings`.

Генерация матрицы реализована в классе `RandomMatrixGenerator`. Функция генерация матрицы из этого класса запускается при нажатии соответствующей кнопки в главном меню. Для чтения матрицы из файла используется класс `MatrixReader`, в который передается путь, получение которого описано в п. 2.4; затем вызывается функция, считывающая матрицу значений. Передача значений после задания матрицы вручную реализовано в цикле после нажатия кнопки `continue` в меню задания матрицы. Числа из ячеек с выбором числа передаются в `costMatrix`.

После генерации матрицы случайным образом, загрузки ее из файла или записи вручную алгоритм запускается в фоновом режиме в окне loading_screen для расчета и отображения процесса работы (рис. 11).

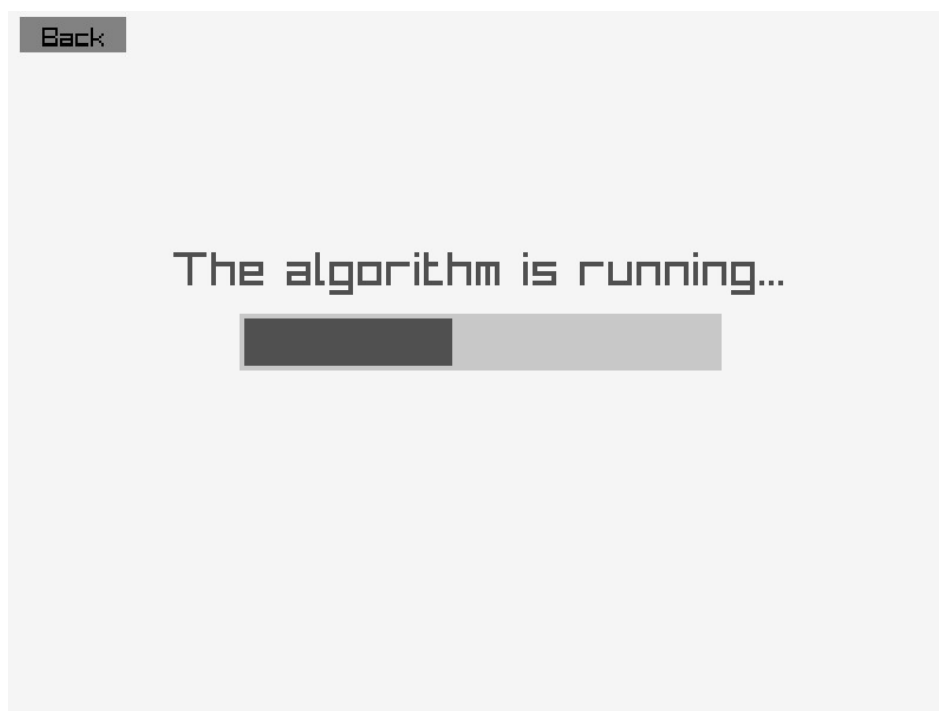


Рисунок 11 — окно с ожиданием работы алгоритма.

Затем результат сохраняется в result, открывается окно визуализации (рис. 5) и пользователь может наблюдать шаги работы алгоритма.

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

Язык программирования, используемый для алгоритма и графического интерфейса: C++ со стандартной библиотекой.

Библиотека для создания графического интерфейса а также визуализации решения задачи: `raylib`. Из библиотеки использовались функции отрисовки линий, окружностей, прямоугольников, а также функции считывания нажатия клавиш и кнопок мыши.

Библиотека для вызова диалога выбора файла: `tinyfiledialogs`. Использовалась для удобного выбора файла матрицы стоимостей из главного меню.

4 ОТЗЫВ РУКОВОДИТЕЛЯ

По результатам работы учебная практика Будкина Вадима оценена на

...

По результатам работы учебная практика Мигрина Данилы оценена на

...

По результатам работы учебная практика Бегазы Андрея оценена на ...

ЗАКЛЮЧЕНИЕ

По результатам практики была достигнута цель. В.А. Будкин реализовал генетический алгоритм, решающий задачу о назначениях, с возможностью настройки параметров алгоритма: метод отбора, метод мутации, метод скрещивания, количество поколений, количество поколений без улучшений и т. д. Д.А. Мигрин реализовал графический интерфейс для удобной настройки алгоритма с помощью библиотеки `raylib`: было реализовано главное меню с полями для настроек, меню задания матрицы вручную, возможность подключения матрицы из файла формата `.csv` с помощью библиотеки `tinyfiledialogs`. С помощью `raylib` А.И. Бегеза реализовал пошаговую визуализацию решения задачи, используя классы в языке `C++`, связал алгоритм и графическую часть программы: взаимодействуя с кнопками пользователь может просматривать разные шаги и решения задачи.